

How Python Dictionaries Work: Internal Mechanism & Hash Tables

Table of Contents

1. Introduction: Dictionaries vs Hash Tables
2. Hash Tables: The Foundation
3. Hash Function Process
4. Lookup Process
5. Hash Collisions and Resolution
6. Resizing and Load Factor
7. Python 3.6+ Optimization
8. Deletion and DEADSLLOT
9. Key Requirements: Hashability
10. Performance Characteristics
11. Real-World Example
12. Why Dictionaries are So Fast
13. Key Takeaways and References

1. Introduction: Dictionaries vs Hash Tables

Python dictionaries are **not just simple data structures**—they are sophisticated implementations of **hash tables**, one of the most important data structures in computer science.[web:99][web:105] Understanding how they work internally helps you write more efficient code, optimize performance, and debug complex issues.

What's the Difference?

In computer science, a **dictionary** is an abstract data type—a specification of operations:

- Add a key-value pair
- Delete a key-value pair
- Update a key-value pair
- Find a value by key

A **hash table** is a concrete data structure—an implementation strategy using hashing to solve the "dictionary problem." [web:99][web:113]

Python's dict is a hash table implementation of the dictionary concept, optimized for fast lookups.

2. Hash Tables: The Foundation

At its core, a dictionary is built on a **hash table**, which consists of three main components:
[web:96][web:97]

Core Components

Hash Function: Converts a key into a numerical index (hash code)

- Takes arbitrary key of any type
- Returns fixed-size integer
- Deterministic (same key → same hash)

Hash Table (Array): Contiguous block of memory with predefined slots

- Fixed-size initially (8 slots for empty dict)
- Stores entries: <hash_value, key, value>
- Random-access data structure

Collision Resolution: Strategy to handle multiple keys mapping to same index

- Python uses **open addressing** with **pseudo-random probing**
- Probes for next available slot if collision occurs

Initial Structure

When you create an empty dictionary, Python initializes it with **8 empty buckets/slots**:
[web:97][web:99]

```
d = {}
```

Internally creates:

indices = [None, None, None, None, None, None, None, None] (8 slots)

entries = [] (compact storage for Python 3.6+)

Each slot can store one entry or mark it as deleted with a **DEADSLLOT sentinel**.

3. Hash Function Process

Let's trace what happens when you insert data:

```
my_dict = {}  
my_dict['apple'] = 5
```

Step 1: Compute Hash

```
hash_value = hash('apple')
```

Result: large integer like 7284271176804795

The hash function is deterministic—same input always produces same hash.

Step 2: Calculate Index

```
table_size = 8  
index = hash_value % table_size  
index = 7284271176804795 % 8 = 5
```

Maps to slot 5 in the hash table

Step 3: Store Entry

```
hash_table[5] = (7284271176804795, 'apple', 5)
```

The entry includes the hash value for verification during collision handling.

Important: Hash Randomization

Different Python interpreter session

```
hash('apple') # Returns different value!
```

Example: 440669153173126140 (in another session)

Python enables **hash randomization** by default for security, making hash values unpredictable across interpreter sessions.^{[web:99][web:105]} You can disable it with PYTHONHASHSEED:

```
PYTHONHASHSEED=1 python -c "print(hash('apple'))"
```

Always returns same hash with fixed seed

4. Lookup Process: Finding Values

When you retrieve a value:

```
value = my_dict['apple']
```

Step 1: Compute Hash Again

```
h = hash('apple') # Same value as insertion
```

Step 2: Calculate Index

```
index = h % 8 # Maps to slot 5
```

Step 3: Verify Match

```
if hash_table[5].hash == h AND hash_table[5].key == 'apple':  
    return hash_table[5].value # Returns 5
```

Both conditions must match:

- Hash values match (quick check)
- Keys are equal (full comparison)

Time Complexity: $O(1)$ average case via direct array indexing.[web:99][web:103]

Lookup with Missing Key

```
value = my_dict['grape'] # Key doesn't exist
```

Step 1: Compute hash → index = 7

Step 2: Check slot 7 → None (empty)

Step 3: Raise KeyError

Reaches empty slot → no probing needed

Finding a missing key is also $O(1)$ on average because probing stops at first empty slot.

5. Hash Collisions and Resolution

What happens when two different keys hash to the same index?

Hypothetical scenario:

```
hash('apple') % 8 = 3  
hash('grape') % 8 = 3 # COLLISION!
```

Both keys want slot 3. Python uses **open addressing with pseudo-random probing** to resolve this.[web:97][web:109]

Collision Resolution Process

Initial Attempt

Both keys hash to index 3

Check slot 3

```
if hash_table[3].key == 'apple':  
    return hash_table[3].value  
if hash_table[3].key == 'grape':  
    # No match - start probing
```

Probe to Next Slot

Python uses **quadratic probing formula**:
$$\text{new_index} = (5 * \text{current_index} + 1) \% \text{table_size}$$

Starting from index 3, try: 0, 2, 7, 1, etc.

Probe sequence from index 3:

```
probe_index = (5 * 3 + 1) % 8 = 0 # Try slot 0  
probe_index = (5 * 0 + 1) % 8 = 1 # Try slot 1  
probe_index = (5 * 1 + 1) % 8 = 6 # Try slot 6
```

... continue until finding empty slot or match

Store at Available Slot

Find slot 6 is empty

```
hash_table[6] = (hash_grape, 'grape', 12)
```

Result

```
hash_table = [  
    None, # 0  
    None, # 1  
    None, # 2  
    (hash_apple, 'apple', 5), # 3  
    None, # 4  
    None, # 5  
    (hash_grape, 'grape', 12), # 6  
    None # 7  
]
```

Key Points on Collisions

Collision $\neq$ Data Loss: Both entries are stored, just in different slots[web:97][web:106]

Both Hash and Key Must Match: When probing, Python verifies both:

- Hash values match
- Key objects are equal

This handles the rare case where different types have same hash (e.g., 1 and 1.0).

Uniform Distribution: A good hash function minimizes collisions by distributing hash values uniformly across the table.[web:103][web:115]

6. Resizing and Load Factor

As you add entries, Python monitors the **load factor**: [web:99][web:105]

$\text{load_factor} = \text{number_of_entries} / \text{table_size}$

When Resizing Occurs

When $\text{load_factor} > 2/3$, the hash table **doubles in size**: [web:99][web:105]

Original state

entries: 5
table_size: 8
load_factor: $5/8 = 0.625$ (62.5% full)

Add 1 more entry

load_factor: $6/8 = 0.75$ (75% full) $\rightarrow$ RESIZE TRIGGERED!

After resizing

table_size: 16 (doubled)
load_factor: $6/16 = 0.375$ (37.5% full)

Resizing Process

1. **Create new, larger hash table** (double size)
 $\text{new_table_size} = 16$
 $\text{new_hash_table} = [\text{None}] * 16$
2. **Recompute hash for EVERY existing entry** with new table size
 for entry in old_hash_table:
 $\text{new_index} = \text{entry.hash} \% 16$ # Different from hash % 8!
 $\text{new_hash_table}[\text{new_index}] = \text{entry}$
3. **Move all entries to new positions**
 - Since $\text{hash} \% 16 \neq \text{hash} \% 8$, entries scatter to new locations

- Collisions may reduce due to larger table
4. **Delete old table**
old_hash_table = new_hash_table

Why Resize?

Maintaining Collision Rates: Keeping load factor below 2/3 minimizes collisions[web:99][web:105]

Preserving O(1) Performance: Without resizing, collision chains grow, degrading performance to O(n)[web:105]

7. Python 3.6+ Optimization: Compact Layout

Prior to Python 3.6, dictionaries wasted significant memory. The new layout uses a **two-array system**: [web:99][web:105][web:108]

Old Layout (Pre-3.6)

Sparse single array with 24-byte entries each:

[<entry>, EMPTY, <entry>, EMPTY, EMPTY, <entry>, EMPTY, EMPTY]
24 bytes each

Problem: Many empty slots reserved for future growth = memory waste[web:99]

New Optimized Layout (3.6+)

Sparse index table (just stores indices)

indices = [None, 1, None, None, 0, None, None, 2]
Pointers to entries in compact array
None = empty, 0/1/2 = entry positions

Dense entries table (compact, insertion-ordered)

```
entries = [  
    (hash1, 'apple', 5), # Index 0 (1st insertion)  
    (hash2, 'banana', 3), # Index 1 (2nd insertion)  
    (hash3, 'cherry', 7) # Index 2 (3rd insertion)  
]
```

Benefits of Compact Layout

58% Less Memory: 192 bytes → 90 bytes for 3-item dict[web:108]

Insertion Order Preserved: Entries array maintains insertion order naturally[web:108]

Faster Iteration: No empty slots to skip during iteration[web:99][web:108]

Better Cache Locality: Contiguous entries array fits better in CPU cache[web:108]

8. Deletion: Handling Deleted Items

Deleting entries is tricky—naive deletion breaks probing chains:

```
hash_table = [  
    None,  
    (h1, 'a', 1),  
    None,  
    (h2, 'b', 2),  
    None  
]
```

Delete 'a' naively:

```
hash_table = [None, None, None, (h2, 'b', 2), None]
```

Problem: Searching for 'c' that probed past 'a' fails!

Probing stops at empty slot before finding 'c'

Solution: DEADSLLOT Sentinel

Mark deleted entries with a **DEADSLLOT** sentinel instead of removing them:[web:105]

```
DEADSLLOT = object() # Special sentinel object
```

```
hash_table = [  
    None,  
    DEADSLLOT, # Marks 'a' as deleted  
    None,  
    (h2, 'b', 2),  
    None  
]
```

Now probing continues past DEADSLLOT to find 'c'

How It Works

```
def lookup_with_probing(key):  
    index = hash(key) % len(table)
```

```
    for i in range(len(table)):  
        slot = table[index]  
  
        if slot is None:          # Empty slot - stop searching  
            raise KeyError(key)  
  
        if slot is DEADSLLOT:    # Deleted slot - keep probing  
            index = (5 * index + 1) % len(table)  
            continue  
  
        if slot.key == key:      # Match found  
            return slot.value  
  
        index = (5 * index + 1) % len(table) # No match - probe next
```

Cleanup During Resizing

When the hash table resizes, DEADSLLOT entries are completely removed during rehashing:

Old table with DEADSLLOT entries

```
old_table = [None, DEADSLLOT, (h1, 'a', 1), DEADSLLOT, None]
```

After resize - DEADSLLOT entries are gone

```
new_table = [None, (h1, 'a', 1), None, None, ...]
```

9. Key Requirements: Hashability

For a key to work in a dictionary, it must be **hashable**.^[web:97]^[web:99]

Contract Between Hash and Equality

Fundamental rule:

```
if a == b:  
    hash(a) == hash(b) # Same keys → same hash
```

Contrapositive:

```
if hash(a) != hash(b):  
    a != b # Different hashes → different keys
```

Implementing Custom Hashable Type

```
class Person:  
    def init(self, name, age):  
        self.name = name  
        self.age = age
```

```
    def __hash__(self):  
        # Return consistent integer based on immutable attributes  
        return hash((self.name, self.age))  
  
    def __eq__(self, other):  
        # Used to verify collision matches  
        if not isinstance(other, Person):  
            return False  
        return self.name == other.name and self.age == other.age
```

Now usable as dictionary key

```
people = {Person("Alice", 30): "Developer"}
```

Why Mutability Prevents Hashing

Immutable types are hashable

```
hash((1, 2, 3)) # Works - tuple doesn't change  
hash("hello") # Works - string doesn't change  
hash(42) # Works - int doesn't change
```

Mutable types are NOT hashable

```
hash([1, 2, 3]) # TypeError: unhashable type: 'list'  
hash({1: 2}) # TypeError: unhashable type: 'dict'
```

Why? If you modify the list, its hash changes

But the hash table can't find it anymore!

```
my_list = [1, 2, 3]  
d = {my_list: "value"} # TypeError  
my_list.append(4) # Changes the list
```

**Now where should the entry go?
Impossible to track!**

10. Performance Characteristics

Operation	Time	Why
Access <code>d[key]</code>	O(1) avg	Direct hash-to-index lookup
Insert <code>d[key] = val</code>	O(1) avg	Hash computation + direct insert
Delete <code>del d[key]</code>	O(1) avg	Hash lookup + mark DEADSLOT
Iteration for <code>k</code> in <code>d</code>	O(n)	Must visit all <code>n</code> entries
Lookup missing	O(1) avg	Stops at first empty slot
Collision chain lookup	O(1) avg	Pseudo-random probing scales well

Worst Case: O(n)

Pathological case with many hash collisions (rare in practice):

```
class BadHash:
def init(self, value):
self.value = value
```

```
def __hash__(self):
    return 0 # All objects hash to 0!

def __eq__(self, other):
    return self.value == other.value
```

```
d = {BadHash(i): i for i in range(1000)}
```

Every lookup probes through ~500 entries on average

O(n) behavior in practice

Real-world: Python's hash function and resizing strategy make O(n) extremely unlikely.
[web:99][web:103]

11. Real-World Example: Step Through

```
d = {}  
d['x'] = 10  
d['y'] = 20  
d['z'] = 30
```

State After Each Insertion

Step 1: Insert ('x', 10)

indices: [None, 0, None, None, None, None, None, None]
(at position for hash('x') % 8)
entries: [(hash_x, 'x', 10)]

Step 2: Insert ('y', 20)

indices: [1, 0, None, None, None, None, None, None]
(at position for hash('y') % 8)
entries: [
(hash_x, 'x', 10),
(hash_y, 'y', 20)
]

Step 3: Insert ('z', 30)

indices: [1, 0, 2, None, None, None, None, None]
(at position for hash('z') % 8)
entries: [
(hash_x, 'x', 10),
(hash_y, 'y', 20),
(hash_z, 'z', 30)
]

Key Observations

1. **Entries array is compact:** No gaps, insertion-ordered
 2. **Indices array is sparse:** Contains only positions in entries array
 3. **Fast lookup:** Direct index computation + array access
 4. **Memory efficient:** Only stores actual entries (3) + sparse index (8)
 5. **Iteration order:** Follows entries array order (insertion order)
-

12. Why Dictionaries are So Fast

1. O(1) Average Access

Hashing provides direct array index → no searching needed

2. Smart Resizing

Maintains collision rates below 2/3 load factor automatically[web:99]

3. Efficient Probing

Pseudo-random probing finds empty slots quickly[web:105]

4. Caching Benefits

Dense memory layout (Python 3.6+) fits better in CPU cache[web:108]

5. No Sorting Required

Unlike sorted structures (B-trees, etc.), no comparison overhead

6. Optimized Hash Function

Python uses SipHash24 for strings—fast and collision-resistant[web:99]

```
import sys
print(sys.hash_info.algorithm) # 'siphash24'
```

13. Key Takeaways

1. **Dictionaries = Hash Tables:** Maps keys to values via hashing
2. **Hash Function:** Converts key to index in O(1) time[web:97][web:99]
3. **Collisions:** Resolved using open addressing + pseudo-random probing[web:97][web:109]
4. **Load Factor:** Dictionaries auto-resize when >2/3 full[web:99][web:105]
5. **Python 3.6+:** Compact two-array layout saves 58% memory[web:108]
6. **Hashability:** Keys must be immutable + implement `__hash__` and `__eq__`[web:97]
7. **O(1) Average Performance:** Why dicts are fundamental to Python[web:99][web:103]
8. **Hash Randomization:** Security feature preventing DoS attacks[web:99]
9. **Deletion:** Uses DEADSLOT sentinel to maintain probing chains[web:105]
10. **Real-World Impact:** Understanding internals helps optimize code

References

- [1] Real Python. (2023). Build a Hash Table in Python With TDD. <https://realpython.com/python-hash-table/>
- [2] GeeksforGeeks. (2021). Internal Structure of Python Dictionary. <https://www.geeksforgeeks.org/python/internal-structure-of-python-dictionary/>

- [3] Stack Overflow. (2019). How are Python's Built In Dictionaries Implemented? <https://stackoverflow.com/questions/327311/how-are-pythons-built-in-dictionaries-implemented>
- [4] Stack Overflow. (2017). Handling Hash Collisions in Python Dictionaries. <https://stackoverflow.com/questions/42102163/handling-hash-collisions-in-python-dictionaries>
- [5] Clement Rebuffel. (2021). Python's Dict, from scratch. <https://clement-rebuffel.com/blog/python-dict-from-scratch/>
- [6] TenThousandMeters. (2012). Python behind the scenes #10: How Python Dictionaries Work. <https://tenthousandmeters.com/blog/python-behind-the-scenes-10-how-python-dictionaries-work/>
- [7] DataCamp. (2024). A Guide to Python Hashmaps. <https://www.datacamp.com/tutorial/guide-to-python-hashmaps>
- [8] GeeksforGeeks. (2019). Python 3.6 Dictionary Implementation using Hash Tables. <https://www.geeksforgeeks.org/python/python-3-6-dictionary-implementation-using-hash-tables/>
- [9] Fluent Python. (2011). Internals of sets and dicts. <https://www.fluentpython.com/extra/internals-of-sets-and-dicts/>

Document created for educational purposes—suitable for curriculum development, student understanding, interview preparation, and technical reference.